



Docker Step 4 — Docker's Default Network

How containers find and talk to each other — from the concept down to the bits

Companion to the Step 4 multi-tier app (Next.js → Express → Redis) • Generated 26 June 2026

What this guide is. A deep, beginner-friendly tour of the network Docker quietly creates for every Compose project — the thing that makes `http://backend:4000` magically work. We start from "what is a network," then peel back every layer: drivers, bridges, the `inspect` output, IP addresses, the `/16` notation, gateways, and the private DNS that powers service discovery. Everything is explained from scratch.



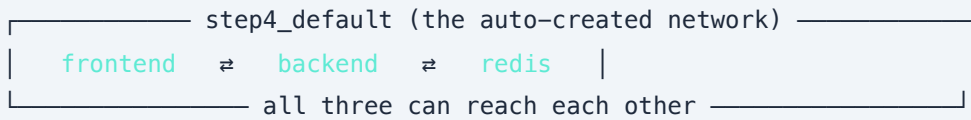
Contents

1. What is a Docker network?
2. "Driver" — the type of network
3. "Bridge" — the virtual switch
4. The networks on your machine (`docker network ls`)
5. Built-in bridge vs Compose bridge (the DNS difference)
6. Looking inside (`docker network inspect`)
7. IP addresses: octets, bits & binary
8. The subnet & the `/16` notation
9. The gateway — the door out
10. Service discovery: how names become IPs
11. Why DNS only works inside the container
12. Commands cheat-sheet
13. Key takeaways & glossary

1. What is a Docker network?

A **Docker network** is a private, virtual network that containers join so they can talk to each other. It's the reason your three services can communicate.

The big idea you must hold onto: **when you run `docker compose up`, Compose automatically creates one network for your project** and connects every service to it. You never asked for it — it just happens. Its name is `<project>_default` (e.g. `step4_default`).



💡 **Why it matters:** this network is what lets your code say `http://backend:4000` instead of a hard-coded IP. Names work *because* of this network.

2. "Driver" — the type of network

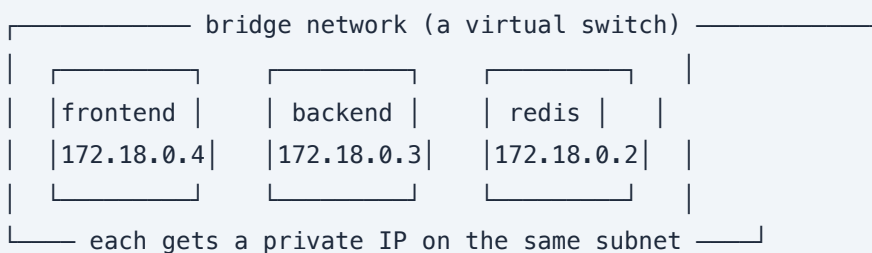
A **driver** is the software that decides *how* a network behaves. You pick one when a network is created. Think of it as choosing a *kind of road*: a highway and a driveway are both roads, but very different.

Driver	What it does	When used
bridge	private virtual network on <i>one</i> machine	the default — everything you've done
host	no isolation; container shares the host's network	rare, special cases
none	no networking at all (fully isolated)	rare, special cases
overlay	a network spanning <i>multiple</i> machines	clusters (Swarm / Kubernetes)

In `docker network ls`, the **DRIVER** column tells you which type each network is. Most say **bridge** because that's the standard for a single machine.

3. "Bridge" — the virtual switch

A **bridge** is one specific driver — and the one your containers use. The name comes from networking hardware: a bridge (or switch) connects several computers into a small local network. Docker does this *in software*.



🏠 **The home-router analogy:** a bridge network is like your home Wi-Fi router. Your phone, laptop, and TV all connect to it and can talk to each other; each gets a private IP; devices on a

different router can't reach yours. Same with containers on a bridge network.

4. The networks on your machine

Running `docker network ls` shows something like:

NETWORK ID	NAME	DRIVER	SCOPE
663aa164019a	bridge	bridge	local
8a6d25b5b0ae	host	host	local
323c8dc5b681	none	null	local
ea7617aef31b	step4_default	bridge	local

Column	Meaning
NETWORK ID	a unique short ID Docker assigns
NAME	the human-friendly name
DRIVER	the network <i>type</i> (§2)
SCOPE	<code>local</code> = lives on this one machine

The first three (`bridge` , `host` , `none`) are **built in** — Docker creates them on install; you can't remove them. The fourth, `step4_default` , is **yours** — Compose made it for your project, and it's the one your containers actually use.

Naming pattern: `step4_default` = `<project>_<network>` . Compose always prefixes networks and volumes with the project (folder) name, so different projects never collide.

5. Built-in bridge vs Compose bridge (the DNS difference)

This is the subtle bit. Both the built-in `bridge` and `step4_default` have driver `bridge` — but they behave differently in one crucial way:

	Built-in <code>bridge</code>	Compose's <code>step4_default</code>
Used by	plain <code>docker run</code> (no network given)	your Compose services
Name-based DNS?	No — reach others only by raw IP	Yes — reach others by service name

💡 **The key insight:** the magic of `http://backend:4000` is *not* a property of bridge networks in general. It's specifically that **Compose creates its own bridge network with DNS built in**. That added DNS is what resolves service names.

6. Looking inside (`docker network inspect`)

`docker network inspect step4_default` prints a lot of JSON. Here are the fields that actually matter:

```
"Driver": "bridge",           // the type
"Scope": "local",           // this machine only
"IPAM": {
  "Subnet": "172.18.0.0/16",  // the pool of addresses (see §8)
  "Gateway": "172.18.0.1"    // the door out (see §9)
},
"Internal": false,          // false = can reach the internet
"Labels": { "...compose.project": "step4" }, // who created it
"Containers": {
  "step4-frontend-1": { "IPv4Address": "172.18.0.4/16" },
  "step4-backend-1": { "IPv4Address": "172.18.0.3/16" },
  "step4-redis-1":    { "IPv4Address": "172.18.0.2/16" }
}
```

Field	What to read from it
IPAM	"IP Address Management" — the subnet + gateway plan
Internal	<code>false</code> = containers can reach the outside world; <code>true</code> = sealed off
Containers	the live membership list — who's connected and at what IP
Labels	confirms Compose made it (project = step4)

Safe to ignore for now: `EnableIPv6` , `Attachable` , `Ingress` , `ConfigFrom/ConfigOnly` , `EndpointID` , `MacAddress` — these are advanced/cluster-only or internal bookkeeping.

7. IP addresses: octets, bits & binary

An address like **172.18.0.4** is **four numbers** (each 0–255) separated by dots. Each number is called an **octet**.

```
172 . 18 . 0 . 4
  |   |   |   |
  |   |   |   |
oct1 oct2 oct3 oct4 ← 4 octets × 8 bits = 32 bits total
```

Why an octet = 8 bits

"Octet" literally *means* eight (**oct** = 8, like **octopus** / **octagon**). An IPv4 address is 32 bits, split into 4 equal parts → **8 bits per part**. So each octet is 8 bits by definition.

Why each octet maxes at 255

A **bit** is a single binary digit (0 or 1). In 8 bits, each position is a power of 2:

```
position: 128 64 32 16 8 4 2 1
172 = 1 0 1 0 1 1 0 0
      = 128 + 32 + 8 + 4 = 172
```

- All 8 bits **1** → **128+64+32+16+8+4+2+1 = 255** (the max)
- All 8 bits **0** → **0** (the min)
- So 8 bits hold **256 values (0–255)** = 2^8 .

💡 **In short:** each number in an IP is 8 bits, and 8 bits can only count from 0 to 255 — that's why you never see 256 or higher in an IP.

8. The subnet & the **/16** notation

172.18.0.0/16 describes the **pool of addresses** the network owns. The **/16** says: "the first 16 bits are the fixed *network* part; the rest is free for devices."

Since each octet is 8 bits, **16 bits = the first two octets**:

```
172 . 18 . 0 . 0 /16
  |_____|  |_____|
```

NETWORK (fixed) HOST (free)
every device starts 172.18 last two octets vary

So this network owns every address from **172.18.0.0** to **172.18.255.255** — about **65,536** addresses (2^{16}). That matches the **inspect** output: ~65,530 free + a few in use.

Notation	Fixed part	Free part	Total addresses	Size
/8	172	.x.x.x	~16 million	huge
/16	172.18	.x.x	~65,000	medium (Docker default)
/24	172.18.0	.x	256	small (typical home Wi-Fi)

Pattern: the slash number = how many bits are locked as the network name. Each step of 8 (**/8** → **/16** → **/24**) locks one more octet. Smaller slash = bigger network.

9. The gateway — the door out

"Gateway": "**172.18.0.1**" is the network's **exit door**, almost always the first usable address (**.0.1**). When a container needs to reach something *outside* its network — the internet, or another network — it sends that traffic to the gateway, which forwards it onward.

container 172.18.0.4 —"I need google.com"—> gateway 172.18.0.1 —> internet

 **Analogy:** exactly like your home router (**192.168.1.1**) — every device sends outbound traffic to it to reach the internet.

10. Service discovery: how names become IPs

"Service discovery" = reaching another container by its **service name** instead of an IP. It's powered by Docker's **embedded DNS server**, which lives at the special address `127.0.0.11` inside every container.

```
# Inside a container, this file points DNS at Docker's resolver:
$ docker compose exec backend cat /etc/resolv.conf
nameserver 127.0.0.11      # ← Docker's internal DNS

# Prove a name resolves (and to which IP):
$ docker compose exec frontend getent hosts backend redis
172.18.0.3      backend
172.18.0.2      redis
```

What `getent hosts <name>` does: "get entries" from the system's `hosts` (name→IP) database. A reply proves the name resolves; **no** output proves it doesn't (useful for testing isolation).

Who calls whom	Address	Why it works
frontend → backend	<code>http://backend:4000</code>	Docker DNS resolves <code>backend</code> → its IP
backend → redis	<code>redis://redis:6379</code>	Docker DNS resolves <code>redis</code> → its IP

💡 **Why names beat IPs:** container IPs can change on restart, but service names never do. Coding against names means you never hard-code a fragile address.

11. Why DNS only works inside the container

The name `backend` is resolved by **Docker's private DNS** (`127.0.0.11`), which only exists inside the Docker network. Containers and your Mac ask *different* resolvers:

INSIDE container:	ON your Mac:
"what is backend?"	"what is backend?"
↓	↓
Docker DNS (127.0.0.11)	System / router DNS
knows: backend, redis ✅	knows: google.com...
↓	never heard of "backend" ❌
↓	
172.18.0.3 ✅	

Docker deliberately keeps service-name DNS **scoped to the network** — it would be chaos if every program on your Mac could resolve project-internal names like `db` or `redis`. This is the *same* reason the **browser** can't reach `http://backend:4000`: the browser uses the Mac's DNS, which doesn't know Docker names.

⚠️ **On macOS, even the IP won't help.** Docker runs inside a Linux VM, so `172.18.x.x` addresses live in that VM, not on your Mac. To reach a container from the Mac you must **publish a port** (e.g. `3000:3000`).

12. Commands cheat-sheet

```
# List all networks
docker network ls

# Inspect one network (members, subnet, gateway)
docker network inspect step4_default

# Pull just the members + IPs
docker network inspect step4_default \
  --format '{{range .Containers}}{{.Name}} {{.IPv4Address}}{{println}}{{end}}'

# From inside a container — prove DNS resolution
docker compose exec frontend getent hosts backend
docker compose exec backend cat /etc/resolv.conf

# Clean up unused networks
docker network prune
```

13. Key takeaways & glossary

- Compose **auto-creates** a bridge network (`<project>_default`) and connects every service to it.
- Service names work because that network has a **built-in DNS** (`127.0.0.11`) — the plain `bridge` network does not.
- Each container gets a private **IP** on the network's **subnet**; the **gateway** is the door to the outside.
- `/16` means the first 16 bits (two octets) are the fixed network part; an octet is 8 bits (0–255).
- DNS works **only inside** the network — the Mac and the browser use a different resolver.

Term	Meaning
Network	a virtual network containers join to talk
Driver	the <i>type</i> of network (bridge, host, none, overlay)
Bridge	a virtual switch on one machine (the default driver)
Subnet	the range of IPs a network owns (e.g. <code>172.18.0.0/16</code>)
Octet	one of the four numbers in an IP = 8 bits = 0–255
CIDR (<code>/16</code>)	how many bits are the fixed network part
Gateway	the exit address to reach outside the network
IPAM	IP Address Management — the subnet/gateway plan
Service discovery	reaching a container by its service name via DNS
Embedded DNS	Docker's resolver at <code>127.0.0.11</code> inside containers

Docker Step 4 — Docker's Default Network • Companion to the multi-tier app (Next.js → Express → Redis)
 The network you never see is the one doing all the work. 🌐